

SCRIPTS

¿Qué son?

¿Qué es un script? Un script es un **“guión”** con instrucciones de código, (básicamente, lo que venimos haciendo hasta ahora) guardado con un nombre y **ejecutado desde el intérprete (ide)**, estos scripts pueden tomar datos (**argumentos**) en el momento de la ejecución.

Pueden tomar estos datos desde el exterior y tener distintos comportamientos.

IDE

Vamos a trabajarlo desde Colabs como siempre. Pero lo más recomendado es utilizar un editor de texto de escritorio.

Más adelante vamos a utilizar VisualStudioCode, sin embargo pueden instalar el que deseen, recomendamos **Notepad++**, **SublimeText**, **IntelliJ**, **Pycharm**.



¿Cómo se crea?

Tenemos que crear un documento de texto, podemos ponerle el nombre que queramos, y después tenemos que cambiar la extensión.

Por lo general los archivos de texto tienen de extensión un **txt** pero tendremos que cambiarlo a un archivo Python, cuya extensión es **py**.

Luego de cambiar la extensión, abrimos el archivo con alguno de los editores descargados.

¿Cómo se crea?



Dentro del archivo simplemente pondremos un: `print("Hola mundo, este es mi primer script en python")`

Y lo guardamos en nuestro drive:



programa.py



Luego lo llamamos desde nuestro Colabs:

```
[1]: from google.colab import drive
    drive.mount('/content/drive')

Mounted at /content/drive

!python "/content/drive/MyDrive/238"
```

SCRIPTS CON ARGUMENTOS

¿Qué son?

Ahora sabemos como crear scripts, pero la idea era tener una entrada de datos, no?

Los datos que se van a enviar se denominan **argumentos**, y son valores que se pasan al ejecutar un script para que este mismo después los modifique o haga lo que programemos con estos datos.

Pasar Argumentos a un Script

Mediante consola (cmd), lo pasamos como si fuera un texto extra a añadir:

```
!python primer_script.py argumentos
```

Todo se enviará como texto, pero, podemos enviar varios valores.

Enviar Strings

En este caso, primero debemos llamar a nuestro script separado por espacios y entre comillas, de la siguiente forma:

!python

primer_script.py

“Cadena”

5

[1,2,3,4]



Scripts con Argumentos

Tenemos que ir al script y editarlo. Primero importamos una librería llamada **sys** y después imprimimos los argumentos usando la librería.

```
import sys  
print(sys.argv)
```



Scripts con Argumentos

Al ejecutar el script podemos ver que devuelve una lista con una cadena que contiene el nombre del script y los argumentos que le pasamos. Siempre el **primer argumento** de la lista **`sys.argv` (`sys.argv[0]`)** es el propio **nombre del script** y lo siguiente son los argumentos que hemos mandado, como **string, int** y **list** respectivamente.

```
['primer_script.py', "'hola'", '1', '[1,2,3]']
```

Script con Argumentos



```
import sys

# Comprobación de seguridad, ejecutar sólo si se reciben 2 argumentos reales
if len(sys.argv) == 3:
    texto = sys.argv[1]
    repeticiones = int(sys.argv[2])
    for r in range(repeticiones):
        print(texto)
else:
    print("Error - Introduce los argumentos correctamente")
    print('Ejemplo: escribir_lineas.py "Texto" 5')
```



Scripts con Argumentos

Al ejecutar el script podemos ver que devuelve una lista con una cadena que contiene el nombre del script y los argumentos que le pasamos. Siempre el **primer argumento** de la lista `sys.argv` (`sys.argv[0]`) es el propio **nombre** del **script**.

```
['primer_script.py', "'hola'", '1', '[1,2,3]']
```



Scripts con Argumentos

Haremos un script que printee un argumento las veces que se indique por argumento, es decir, si paso los argumentos **“Hola” y 5, se imprimirá “Hola” 5 veces.**

Empecemos creando un nuevo archivo llamado ***repeticion.py*** y editemoslo.



Scripts con Argumentos

Si escribimos el siguiente script estaría funcionando:

```
import sys
cadena = sys.argv[1]
repeticiones = int(sys.argv[2])
for repeticion in range(repeticiones):
    print(cadena)
```



Nota: Si no pasamos los argumentos que pide el script, nos va a fallar. Para eso debemos comprobar que existen los argumentos o comprobar la longitud de la lista validandolo con if-else

MÓDULO

¿Qué es?

Un módulo es un archivo que consta de código Python. Puede definir funciones, clases y variables. También puede incluir código ejecutable. Se trata de un archivo con extensión `.py` o `.pyc` (Python compilado) que posee su propio espacio de nombres y que puede contener variables, funciones, clases e incluso otros módulos.

¿Para qué sirve?

Un módulo permite organizar lógicamente el código Python. Agrupar el código relacionado en un módulo hace que el código sea más fácil de entender y usar. Un módulo es un objeto de Python con atributos de nombres arbitrarios que puede enlazar y luego referenciar.

¿Para qué sirve?

Sirve para organizar y reutilizar el código (modularización y reutilización). Se genera uno creando un archivo con extensión `.py` (o `.pyc` o archivo en C) y guardándolo donde nos interese.

Básicamente utilizaremos módulos para organizar y reutilizar nuestro código.

CREACIÓN DEL MÓDULO



Paso a paso



Y pasamos a un IDE

Visual Studio Code.



Paso a paso

1. Creamos una carpeta, donde más cómodo te sientas.
2. Abrimos nuestro editor (VCS) y entramos a la carpeta del punto 1.
3. Creamos un archivo con la extensión .py. En el ejemplo se llamará **funciones_matematicas.py**.
4. Creamos algunas funciones simples en ese módulo, veamos mi

ejemplo

```
EXPLORER
...
Welcome
funciones_matematicas.py

OPEN EDITORS
Welcome
funciones_matematica...

MODULOS
funciones_matematicas...

funciones_matematicas.py > restar
1 def sumar(n1, n2):
2     print (f"La suma me da: {n1+n2}")
3
4 def restar(n1, n2):
5     print (f"La resta me da: {n1-n2}")
```



Hasta ahora parece que no es nada nuevo, ¿Verdad?

Efectivamente es lo mismo de siempre. Pero, aquí viene lo nuevo e interesante: **Vamos a llamar a esas funciones desde otro archivo.**





Paso a paso

5. Crear otro archivo .py, se llamara **uso.py**

6. *Ahora viene lo nuevo*, si queremos que desde uso.py se puede acceder a todo lo realizado en funciones_matematicas.py tenemos que escribir arriba de todo:

```
import funciones_matematicas
```



Paso a paso

Ya casi estamos, pero no alcanza con solo escribir **import**
funciones_matematicas, luego cada vez que quieras hacer referencia a algo de ese módulo deberás escribir antes de un punto **funciones_matematicas**, análogo a como hacíamos con los objetos.

```
uso.py
1  import funciones_matematicas
2
3
4  funciones_matematicas.sumar(4,11)
```



Paso a paso

Ahora al ejecutar, estarás viendo el resultado de la suma o de cualquier otra función que hayas creado en tu módulo. Para evitar escribir a cada rato el nombre del módulo, hay una alternativa muy útil: Podemos reemplazar el **import** por:

```
from funciones_matematicas import suma
```





Paso a paso

La opción anterior, es una buena alternativa cuando son pocas las funciones que a usar de un módulo.

Si vamos a usar muchas, o todas, será mejor que usemos:

```
from funciones_matematicas import *
```



Creando un Módulo

Primero generamos un archivo **atleta.py**, donde ese será nuestro

The image shows a VS Code editor with two Python files open. The left file, `atleta.py`, defines a `Ciclista` class. The right file, `uso.py`, imports the `Ciclista` class and creates an instance. The bottom panel shows the terminal output of the program.

atleta.py

```
1  ##### CICLISTA #####
2  class Ciclista:
3
4      #Constructor, con atributos privados
5      def __init__(self, estilo):
6          self.__estilo = estilo
7
8      #Metodo de este padre
9      def hacerBici(self):
10         print("Estoy andando en bici o--to")
11
12     #Como quiero mostrar al objeto
13     def __str__(self):
14         return f"Soy un ciclista del estilo {self.__estilo}"
15
16     #Getter y setter
17     def get_estilo(self):
```

uso.py

```
1  import funciones_matematicas
2  from atleta import *
3
4  funciones_matematicas.sumar(4,11)
5
6
7
8
9  atleta_nuevo = TriAtleta("Mariposa", "500", "Urbano", "BMX", "Nicolas Perez", 31)
10
11  print(atleta_nuevo)
```

Terminal Output:

```
Soy un nadador de: Mariposa
Soy un corredor de: 500 del tipo 500
Soy un ciclista del estilo BMX
Soy Genial
Mi nombre es Nicolas Perez con 31 años...
PS C:\Users\nico\Desktop\modulos>
```



Creando un Módulo

Luego creamos un **Atleta** desde uso.py.

The screenshot shows a Python IDE with three main panels: a file explorer on the left, a code editor in the center, and a terminal at the bottom.

File Explorer (Left): Shows a project structure with two groups. Group 1 contains 'atleta.py'. Group 2 contains 'Welcome', 'funciones_mate...', 'uso.py' (selected), and 'atleta.py'. Below this, a 'MODULOS' section lists '._pycache_', 'atleta.py', 'funciones_mate...', and 'uso.py'.

Code Editor (Center): Displays the code for 'atleta.py' and 'uso.py'.
atleta.py:

```
1 ##### CICLISTA #####  
2 class Ciclista:  
3  
4     #Constructor, con atributos privados  
5     def __init__(self, estilo):  
6         self._estilo = estilo  
7  
8     #Metodo de este padre  
9     def hacerBici(self):  
10        print("Estoy andando en bici o--to")  
11  
12    #Como quiero mostrar al objeto  
13    def __str__(self):  
14        return f"Soy un ciclista del estilo {self._est  
15  
16    #Getter y setter  
17    def get_estilo(self):
```


uso.py:

```
1 import funciones_matematicas  
2 from atleta import *  
3  
4  
5 funciones_matematicas.sumar(4,11)  
6  
7  
8  
9 atleta_nuevo = TriAtleta("Mariposa", "500", "Urbano", "BMX", "Nicolas Perez", 31)  
10  
11 print(f"{atleta_nuevo}")
```

Terminal (Bottom): Shows the output of running the code:

```
Soy un nadador de: Mariposa  
Soy un corredor de: 500 del tipo 500  
Soy un ciclista del estilo BMX  
Soy Genial  
Mi nombre es Nicolas Perez con 31 años...  
PS C:\Users\nico\Desktop\modulos>
```



Creando un Módulo

Así quedó nuestra carpeta inicial, la que hicimos en el paso 1:

Nombre	Fecha de modificaci
 __pycache__	26/8/2021 19:35
 atleta	26/8/2021 19:32
 funciones_matematicas	26/8/2021 19:15
 uso	26/8/2021 19:35

PAQUETE

Definición



Guardar todos los módulos en la misma “carpeta”
puede ser muy tedioso y difícil de administrar.

¡Imaginen si tienen 100 módulos, esa carpeta estará
plagada de archivos .py. !

Para eso aparecen los **paquetes**, no son otra cosa que **carpetas para
organizar los módulos, por categorías.**

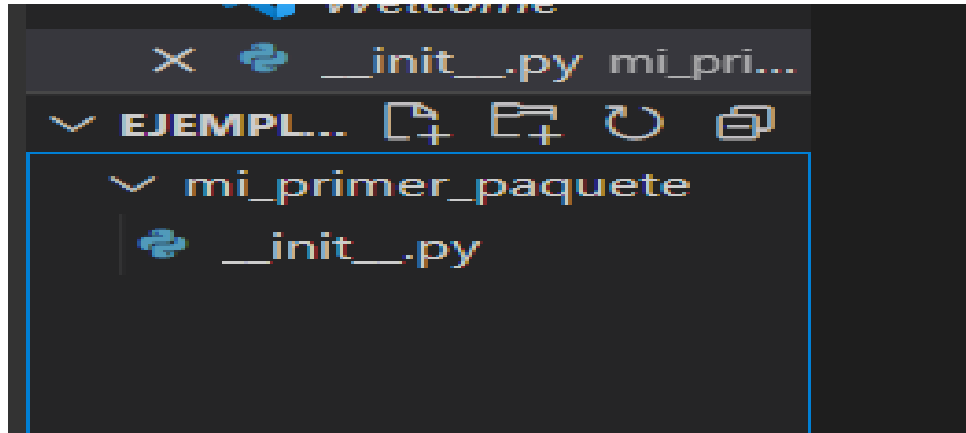
☞ Crear un paquete es muy sencillo, solo tienes que crear una carpeta y
dentro de ella deberas tener un archivo `__init__.py`

Nuestro primer paquete



Paso a paso

1. Podemos crear una carpeta llamada “mi_primer_paquete” con un archivo `__init__.py`





Paso a paso

2. Ahora hagamos dos módulos en el paquete.

The screenshot shows a code editor with two files open in the 'mi_primer_paquete' directory. The left pane shows the file explorer with 'modulo1.py' and 'modulo2.py' selected. The right pane shows the code for 'modulo1.py' and 'modulo2.py'.

```
mi_primer_paquete > modulo1.py > ...
1 class Persona:
2
3     def __init__(self, nombre, apellido):
4         self.nombre = nombre
5         self.apellido = apellido
6
7
8     def __str__(self):
9         return f"NOMBRE {self.nombre}"
10
11 #OJO esto es el primer ejemplo, siempre es aconsejable
12 #modulos y paquetes tengan nombres representativos
```

```
mi_primer_paquete > modulo2.py > llamado_modulo2
1 def llamado_modulo2():
2
3     print("Acá estoy usando el modulo 2 :) ")
```



Paso a paso

Solo nos falta llamar a alguno de nuestros módulos del paquete

`“mi_primer_paquete”.`

3. Para esto creamos un archivo en el `directorio raiz`, lo llamaremos

`“menu.py”.`

4. Importamos al paquete y los módulos de la siguiente manera:

```
from mi_primer_paquete.modulo1 import Persona
```

```
from mi_primer_paquete.modulo2 import llamado_modulo2
```



The screenshot shows a Python IDE interface. On the left is a file explorer with a tree view. It has two groups: 'GROUP 1' containing 'Welcome', '._init_.py mi...', and 'modulo1.py mi...'; and 'GROUP 2' containing 'modulo2.py mi...'. Below these is a section labeled 'EJEMPL...' with a sub-section 'mi_primer_paquete' containing '._pycache_', '._init_.py', 'modulo1.py', 'modulo2.py', and 'menu.py'. The 'menu.py' file is selected. The main area is a code editor showing the contents of 'menu.py':

```
1 from mi_primer_paquete.modulo1 import Persona
2 from mi_primer_paquete.modulo2 import llamado_modulo2
3
4
5 persona1 = Persona("Nicolas", "Lopez")
6
7 print(persona1)
8
9 llamado_modulo2()
```

At the bottom is a terminal window with tabs for 'PROBLEMS', 'OUTPUT', 'TERMINAL', and 'DEBUG CONSOLE'. The 'TERMINAL' tab is active, showing the command prompt output:

```
PS C:\Users\nico\Desktop\ejemploPaquetes> & C:/Users/nico/AppData
menu.py
NOMBRE Nicolas
Acá estoy usando el modulo 2 :)
PS C:\Users\nico\Desktop\ejemploPaquetes> []
```

Así nos queda nuestro código



Paquetes redistribuibles

Definición

Es básicamente un paquete para compartir y que lo pueda usar cualquier otro desarrollador o incluso tu mismo en otro proyecto.



Esto nos permite poder llamar a un paquete sin importar su ubicación.

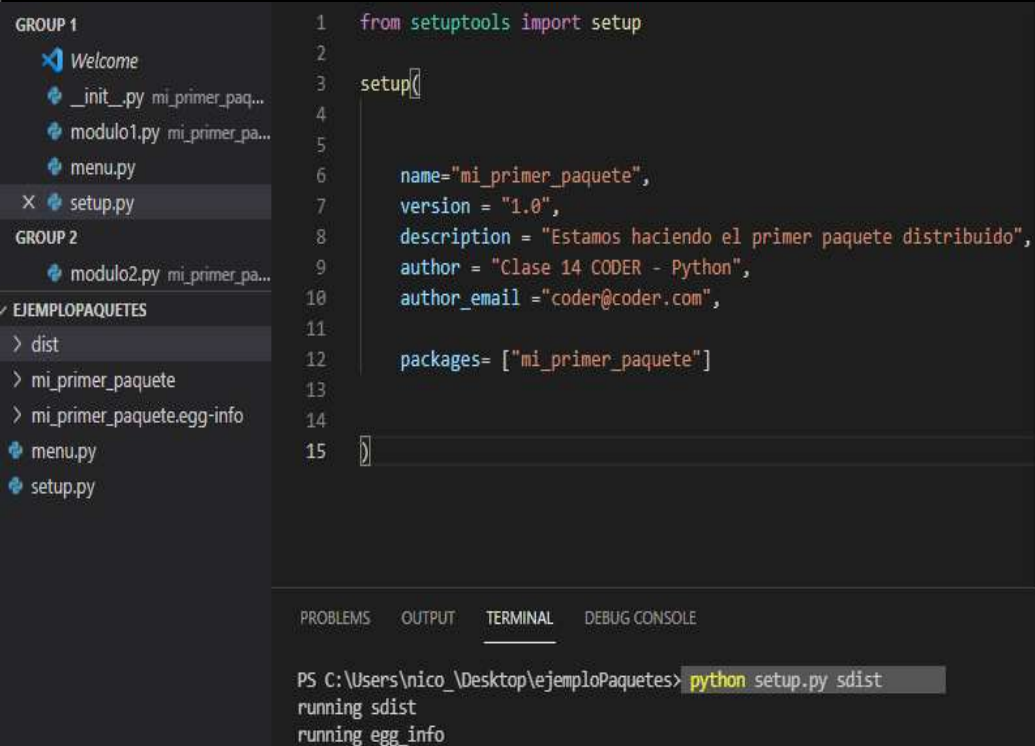


Veamos a continuación

Llamar a un paquete sin importar su ubicación

1- Crear en el directorio raíz un archivo `setup.py` con algunos datos del paquete y luego desde la terminal ejecutar `python setup dist`.

2- Se crean dos carpetas, en la carpeta `dist` tendremos nuestro paquete para compartir.



The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project structure with a 'dist' folder and a 'mi_primer_paquete' folder. The code editor displays the content of 'setup.py', which includes metadata like name, version, description, author, and email, and a list of packages. The terminal shows the command 'python setup.py sdist' being executed, with the output 'running sdist' and 'running egg_info'.

```
1 from setuptools import setup
2
3 setup(
4
5     name="mi_primer_paquete",
6     version = "1.0",
7     description = "Estamos haciendo el primer paquete distribuido",
8     author = "Clase 14 CODER - Python",
9     author_email = "coder@coder.com",
10
11     packages= ["mi_primer_paquete"]
12
13
14
15 )
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

PS C:\Users\nico\Desktop\ejemploPaquetes> python setup.py sdist
running sdist
running egg_info



¿Cómo uso el paquete distribuido de alguien?

Simple, nos ubicamos en el directorio donde tenemos el paquete distribuido que alguien te paso, y ejecutamos:

```
pip3install
```

```
nombrePaquete.tar.gz
```

```
PS C:\Users\nico\Desktop\ejemploPaquetes\dist> pip3 install mi_primer_paquete-1.0.tar.gz
Processing c:\users\nico\Desktop\ejemploPaquetes\dist\mi_primer_paquete-1.0.tar.gz
DEPRECATION: Source distribution is being reinstalled despite an installed package having th
ill remove support for this functionality. A possible replacement is use --force-reinstall.
/pypa/pip/issues/8711.
Using legacy 'setup.py install' for mi-primer-paquete, since package 'wheel' is not installe
Installing collected packages: mi-primer-paquete
  Attempting uninstall: mi-primer-paquete
    Found existing installation: mi-primer-paquete 1.0
    Uninstalling mi-primer-paquete-1.0:
      Successfully uninstalled mi-primer-paquete-1.0
  Running setup.py install for mi-primer-paquete ... done
Successfully installed mi-primer-paquete-1.0
PS C:\Users\nico\Desktop\ejemploPaquetes\dist> |
```

Paquetes/módulos externos

Collections

¿Qué es?



Python 3 tiene varias estructuras de datos integradas, incluyendo tuplas, diccionarios y listas.

Las estructuras de datos nos proporcionan una forma de organizar y almacenar datos. **El módulo collections nos ayuda a completar y manipular las estructuras de datos de forma eficiente.**

Para usarlo debemos importarlo, ya que es un paquete con módulos:

```
from collections import *
```



Namedtuple

Permite añadir nombres explícitos a cada elemento de una tupla para hacer que estos significados sean claros en su programa Python.

👉 Vamos a usar `namedtuple` para generar una clase que claramente denomine a cada elemento de la tupla de peces:

```
from collections import namedtuple
```

```
Fish = namedtuple("Fish", ["name", "species", "tank"])
```



Namedtuple

¡Lo hemos logrado! 🙌

Creamos una clase y la
instanciamos en dos
renglones.

```
1 from collections import namedtuple
2
3 Fish = namedtuple("Fish", ["name", "species", "tank"])
4
5 #Esto crea una clase Fish con los atributos publicos nombre,
6
7 miPrimerPez = Fish("Sammy", "Tiburón", "Tanque grande")
8
9 print(miPrimerPez)
10
11 #otra cosa útil es transforma una instancia de una clase en u
12 #diccionario
13
14 print(miPrimerPez._asdict())
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
PS C:\Users\nico\Desktop\ejemploPaquetes> & C:/Users/nico/AppData/L
/ejemploPaquetes/menu.py
Fish(name='Sammy', species='Tiburón', tank='Tanque grande')
{'name': 'Sammy', 'species': 'Tiburón', 'tank': 'Tanque grande'}
PS C:\Users\nico\Desktop\ejemploPaquetes> □
```

Counter



¿Qué es?

La clase Counter es una subclase de diccionario utilizada para realizar cuentas con diccionarios y listas.

```
16 from collections import Counter
17
18 l = [1,2,3,4,1,2,3,1,2,1]
19 print(Counter(l))
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
PS C:\Users\nico\Desktop\ejemploPaquetes> & C:/Users/nico/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe C:/Users/nico/Desktop/ejemploPaquetes/menu.py
Fish(name='Sammy', species='Tiburón', tank='Tanque grande')
{'name': 'Sammy', 'species': 'Tiburón', 'tank': 'Tanque grande'}
Counter({1: 4, 2: 3, 3: 2, 4: 1})
PS C:\Users\nico\Desktop\ejemploPaquetes> █
```



Ejemplos con texto

Ponemos contar y transformar a diccionario los conteos por letras o palabras.

```
20
21 estudiantes = "Nicolás Claudio Brenda Flor Nicolás Flor"
22 print(Counter(estudiantes))
23
24 print(Counter(estudiantes.split()))
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE Python + - [] [X] ^ >

```
Counter({1: 4, 2: 3, 3: 2, 4: 1})
Counter({'o': 5, 'l': 5, ' ': 5, 'i': 3, 'r': 3, 'N': 2, 'c': 2, 'á': 2, 's': 2, 'a': 2, 'd': 2, 'F': 2, 'C': 1, 'u': 1, 'B': 1, 'e': 1, 'n': 1})
PS C:\Users\nico\Desktop\ejemploPaquetes> & C:/Users/nico/AppData/Local/Microsoft/WindowsApps/python3.9.exe c:/Users/nico/Desktop/
/ejemploPaquetes/menu.py
Fish(name='Sammy', species='Tibrorón', tank='Tanque grande')
{'name': 'Sammy', 'species': 'Tibrorón', 'tank': 'Tanque grande'}
Counter({1: 4, 2: 3, 3: 2, 4: 1})
Counter({'o': 5, 'l': 5, ' ': 5, 'i': 3, 'r': 3, 'N': 2, 'c': 2, 'á': 2, 's': 2, 'a': 2, 'd': 2, 'F': 2, 'C': 1, 'u': 1, 'B': 1, 'e': 1, 'n': 1})
Counter({'Nicolás': 2, 'Flor': 2, 'Claudio': 1, 'Brenda': 1})
```

Otras funciones

Con este paquete podemos, entre otras cosas, también **ordenar, agrupar y rankear registros de un diccionario.**

Datetime

Datetime



```
from datetime import datetime

dt = datetime.now()    # Fecha y hora actual

print(dt)
print(dt.year)         # año
print(dt.month)        # mes
print(dt.day)          # día

print(dt.hour)         # hora
print(dt.minute)       # minutos
print(dt.second)       # segundos
print(dt.microsecond)  # microsegundos
```

Esta es una clase para manejar fechas y horas de una forma muy simple, nos será super útil de acá en adelante.

Por ejemplo: de la siguiente forma podemos acceder al año, mes, día, hora, minuto, segundo y microsegundo del instante en el que creamos una

instancia

Datetime



No solo se puede trabajar con una hora puntual, podemos crear nuestra propia fecha usando el **constructor** de esta clase.

```
from datetime import datetime  
  
dt = datetime(2000,1,1)  
print(dt)
```

```
datetime.datetime(2000, 1, 1, 0, 0)
```

Si queremos cambiar uno de los parámetros, debemos usar la función **replace**.

```
dt = dt.replace(year=3000)  
print(dt)
```

```
datetime.datetime(3000, 1, 1, 0, 0)
```



Personalizar

Es muy sencillo cambiar la forma en la que presentamos un **datetime**

```
dt.strftime("%A %d %B %Y %I:%M")
```

Saturday 18 June 2016 09:37

Otras combinaciones disponibles ➡ [Formateos](#)

Facilidad increíble

Con la función `timedelta()` también puedes sumar o restar tiempos.

```
from datetime import datetime, timedelta

dt = datetime.now()
print(dt.strftime("%A %d de %B del %Y - %H:%M"))

# Generamos 14 días con 4 horas y 1000 segundos de tiempo
t = timedelta(days=14, hours=4, seconds=1000)

# Lo operamos con el datetime de la fecha y hora actual
dentro_de_dos_semanas = dt + t
```

Math

¿Qué es?

Python tiene un módulo incorporado que puede usarse para tareas matemáticas, este módulo se llama **Math**.

A continuación aprenderemos las funciones más importantes.

Pueden verlas a todas desde [math](#)



Algunos usos

```
#Raiz cuadrada
import math

x = math.sqrt(64.74646)

print(f"Raiz cuadrada:{x}")
```

Vamos a importarla con

```
import math
```

```
#Redondeo al entero más cercano

x = math.ceil(1.4)
y = math.floor(1.4)

print(f"Primer redondeo {x}") # ceil
print(f"Segundo redondeo {y}") # floor1
```

Random

¿Qué es?

Este nuevo módulo nos permite obtener valores al azar o aleatorios. Por eso lo conocemos como módulo aleatorio.

A continuación aprenderemos las funciones más importantes.
Pueden verlas a todas desde [random](#)





Algunos ejemplos

Con `import random` llamamos a nuestro módulo aleatorio.

```
#Aleatorios en un rango
print(random.randrange(20, 50))

#Aleatorios en un rango
print(random.randrange(20, 50, 7))
```

42
27

```
lista = [1, 2, "Coder", -1, "Nico", 3]

print(random.choice(lista))
```

Coder

```
string = "Esta es una cadena de caracteres."

print(random.choice(string))
```

t